

Lekce 9: Práce s fixturami a správa testovacích dat

A. Použití fixtur

Proč je správa dat důležitá?

1. Konzistence a spolehlivost:

- **Centralizovaná testovací data:** Ukládání testovacích dat do fixtur nebo externích souborů zajišťuje, že testy probíhají nad konzistentními a kontrolovanými datovými sadami. To snižuje riziko nespolehlivých (flaky) testů způsobených proměnlivými daty.
- **Opakovatelné výsledky:** Se správně spravovanými daty je každý běh testu reprodukovatelný, což usnadňuje diagnostiku problémů při selhání testů.

2. Udržitelnost:

- **Oddělení dat a logiky:** Oddělením testovacích dat od samotných testovacích skriptů je aktualizace či změna vstupních hodnot jednodušší, aniž by bylo potřeba měnit kód testu.
- **Jednodušší aktualizace:** Když se mění datový model aplikace, stačí aktualizovat pouze fixtury nebo datové soubory, ne každý test, který tato data používá.

3. Škálovatelnost:

- **Zpracování více scénářů:** Správa dat umožňuje snadnou parametrizaci testů, takže stejná testovací logika může běžet s různými datovými sadami. To podporuje data-driven testování a napomáhá pokrýt více krajních případů.
- **Jednodušší organizace:** Strukturovaná data podporují lepší organizaci, umožňují seskupování souvisejících testů a spouštění podmnožin dle potřeby.

4. Efektivita automatizace:

- **Mockování externích služeb:** Při testování API pomáhají fixtury simulovat odpovědi backendu, díky čemuž jsou testy nezávislé na živých externích systémech.
- **Dynamická generace dat:** Integrace nástrojů jako Faker umožňuje testům generovat jedinečná data za běhu, vyhnout se konfliktům (např. duplicitní uživatelé) a zajistit, že každý test probíhá v čistém kontextu.

5. Vytváření fixtur (JSON):

- **Definice:**

Fixtury jsou externí datové soubory (obvykle ve formátu JSON), které uchovávají testovací data odděleně od testovacího kódu.

- **Účel:**

- Udržovat testovací data organizovaně a znovupoužitelně.
- Usnadnit údržbu a aktualizaci bez nutnosti měnit samotné testovací skripty.

- **Příklad fixtury:**

Vytvořte soubor `users.json` ve složce `cypress/fixtures` :

```
[
  { "username": "demoUser", "password": "demoPass", "role": "admin" },
  { "username": "userOne", "password": "passOne", "role": "user" }
]
```

6. Načítání fixtur v testech (`cy.fixture()`):

- **Použití:**

`cy.fixture()` načte data z fixtury, která pak můžete použít v testu.

- **Příklad:**

```
cy.fixture('users').then((users) => {
  // Použijte data z fixtury (například pro přihlašovací testy)
  expect(users).to.have.length.above(0);
});
```

7. Strukturování fixtur pro opětovné použití:

- **Nejlepší praktiky:**

- Organizujte fixtury do logických souborů (např. zvlášť uživatelé, produkty, nastavení).
- Používejte jasné a popisné názvy souborů.
- Pokud jsou data hierarchická, správně je vnořte do objektů.

- **Tip:**

Strukturuje fixtury tak, aby odpovídaly různým testovacím scénářům (platná data, neplatná data, krajní případy).

B. Práce s dynamickými daty

1. Generování náhodných dat pro testy:

- **Účel:**

- Testovat, jak aplikace zvládá nepředvídatelné nebo unikátní vstupy.
- Zabránit kolizi dat v testech, které se spouští opakovaně.

- **Postup:**

K generování náhodných řetězců, čísel či dat lze použít JavaScript funkce nebo knihovny (např. `Faker.js`).

- **Příklad:**

```
function getRandomString(length) {
  const chars = 'abcdefghijklmnopqrstuvwxyz';
  let result = '';
  for (let i = 0; i < length; i++) {
    result += chars.charAt(Math.floor(Math.random() * chars.length));
  }
  return result;
}

const randomUsername = `user_${getRandomString(5)}`;
cy.log(randomUsername);
```

Proč je důležité používat Faker?

1. Generování unikátních, realistických testovacích dat:

- **Vyhnutí se kolizím:**
 - Faker generuje náhodná, realistická data (např. jména, emaily, adresy), čímž zamezuje konfliktům jako jsou duplicitní uživatelská jména.
- **Realističnost:**
 - Testovací data podobná reálným umožňují přesnější simulaci uživatelských scénářů.

2. Dynamická data pro parametrizaci:

- **Data-driven testování:**
 - Integrací Fakeru mohou testy dynamicky generovat vstupní data pro každý běh, což umožňuje použít stejnou logiku pro různé vstupy.
- **Škálovatelnost:**
 - Jak vaše testovací sada roste, s Fakerem snadno vygenerujete velké množství dat bez ruční správy statických souborů.

3. Lepší izolace testů:

- **Nezávislé běhy testů:**
 - Díky generování unikátních dat za běhu je menší pravděpodobnost, že se testy navzájem ovlivní. To je zvláště důležité v CI prostředí, kde testy mohou běžet paralelně.

4. Snadné použití:

- **Jednoduché API:**
 - API Fakeru je přímočaré a můžete s ním snadno generovat různé typy dat na pár řádků kódu.
- **Integrace:**
 - Dobře spolupracuje s Cypress; Faker funkce můžete volat přímo ve vašich testovacích skriptech pro generování dat během testování.

Příklad použití Fakeru:

```
// Nainstalujte faker: npm install --save-dev @faker-js/faker
import { faker } from '@faker-js/faker';

const randomUsername = faker.internet.userName();
const randomEmail = faker.internet.email();

cy.log(`Vygenerované uživatelské jméno: ${randomUsername}`);
cy.log(`Vygenerovaný e-mail: ${randomEmail}`);

// Tyto hodnoty použijte v testu, např. pro vytvoření nového uživatele.
cy.get('[data-testid="username-input"]').type(randomUsername);
cy.get('[data-testid="email-input"]').type(randomEmail);
```

2. Parametrizace testů s různými datovými sadami:

- **Použití:**

Spusťte stejnou logiku testu s několika sadami dat z fixtur nebo generovaných dynamicky.

- **Příklad:**

Procházejte pole uživatelů (z fixtury) a spusťte přihlašovací testy:

```
cy.fixture('users').then((users) => {
  users.forEach((user) => {
    cy.get('[data-testid="username-input"]').clear().type(user.username);
    cy.get('[data-testid="password-input"]').clear().type(user.password);
    cy.get('[data-testid="login-button"]').click();
    // Přidejte aserce podle očekávaných výsledků
  });
});
```

2. Praktické aktivity

1. Vytvoření a použití fixtury:

- **Aktivita:**

Vytvořte JSON soubor (např. `users.json`) obsahující několik uživatelských objektů.

Napište Cypress test, který tuto fixturu načte a ověří, že pole uživatelů obsahuje platná data.

2. Dynamické generování dat:

- **Aktivita:**

Napište utilitní funkci, která vygeneruje náhodná uživatelská data (např. uživatelské jméno,

email).

Použijte tato dynamická data v testu k vytvoření nového uživatele a validujte proces.

3. Parametrizace testu:

- **Aktivita:**

Použijte soubor s fixturou pro spuštění stejného přihlašovacího testu pro různé uživatele.

Ověřte, že aplikace reaguje správně pro každý uživatelský scénář.

3. Zdroje

- **Dokumentace k fixturám Cypress:**

[Cypress Fixtures](#)

- **Ukázkové fixtury a příklady testovacích dat:**

Prozkoumejte příklady na GitHubu nebo v dokumentaci Cypress pro praktické použití.

- **Knihovny pro dynamická data:**

Zvažte použití knihoven jako [Faker.js](#) pro generování náhodných testovacích dat.